Execution provenance for human approved application actions

Sanjay Krishna Anbalagan | Individual open-source contributor
Prepared for NIST NCCoE team discussion | 18 October 2026 | For review

I am seeking feedback on a small application-level example that links a recorded human approval to the operation actually attempted. It separates acceptance of an invocation from handler execution and an observed local effect. The example uses synthetic service-request data and deterministic caller fixtures; it does not run a model or connect to an agency system.

Relationship to the NCCoE project

The NCCoE concept paper asks about authority for particular agent actions and records connecting authorization to execution [1]. The stakeholder comment summary discusses auditability beyond action logging and contextual authorization [2]. This example explores one implementation boundary within those questions; it does not implement an identity protocol or establish conformance to a NIST standard.

Implementation and evidence

The run uses published footprintjs 10.1.0 and hcifootprint 3.0.0. HCIFootprint checks approval at the configured agent-invocation boundary; its single-use decision binds an action and its input. FootPrint.js records managed workflow activity. An example-owned adapter connects request, decision, dispatch, handler outcome and before/after state; this is not a native combined feature of the libraries.

A trusted host fixture simulates the human decision through approveAsk; no person is authenticated by this example. Caller-supplied confirmation is refused without that recorded decision. The harness awaits settlement, then checks the application state separately from the returned result. The state check is in the same process, not an independent external observer.

Reproduced scenarios

Scenario	Dispatch and native settlement	Observed local effect
No recorded approval	APPROVAL_REQUIRED	No handler entry; no new revision
Caller supplies confirm:true	APPROVAL_REQUIRED	No handler entry; no new revision
Approval presented for different input	APPROVAL_MISMATCH	No handler entry; no new revision
Approval presented for a different action	APPROVAL_MISMATCH	No handler entry; no new revision
Recorded approval permits one local change	Accepted; settled performed	Handler returned; closed, revision +1
Reusing a consumed approval	APPROVAL_SPENT	No handler entry; no new revision
Handler returns, expected state is absent	Accepted; settled refused: VERIFY_FAILED	Handler returned; no new revision
Handler reports failure after a local state change	Accepted; settled refused	Handler returned; closed, revision +1
Raw application method bypasses the session gate	Session gate not crossed; no fired transition	Handler returned; closed, revision +1

All 9 scenario assertions matched their expected outcomes. This includes deliberate failure and boundary-exposure controls; it is not a security certification or an estimate of real-world attack resistance.

Trust boundaries and limitations

Two counterexamples matter: a no-op reports declared write keys (effectVerified:true) while the application predicate fails (verifyHeld:false). A second handler changes local state and then returns failure without updating the session state projection. Both settle as refused. Write-key observation is not desired-value verification, and refusal does not guarantee rollback.

A production host must authenticate people and callers, protect the approval API and supply principal context. Source labels and the approval by field here are fixtures, not authenticated identities. Integration with identity, delegation and policy-decision systems remains a host responsibility.

The configured agent-invocation path is the enforcement boundary; direct methods and unguarded routes remain outside it. Single-use approval is consumed when dispatch is accepted, before the handler completes. A spent approval therefore does not establish successful execution. The replay fixture measures changes relative to its already-completed setup invocation.

The records cover managed workflow activity and observed in-memory state. They do not prove an external database commit, cryptographic authenticity, tamper resistance, complete capture of all routes, or hidden model reasoning. Local artifact hashes support file-integrity comparison only. The run is a deterministic demonstration, not a formal proof, benchmark or user study.

Questions for the team

- Which existing identity and authorization interfaces should carry host-authenticated actor, delegation and policy-decision references into an application execution record?
- What minimum evidence should distinguish refusal, accepted dispatch, failed execution and an unverified downstream effect?
- Would this bounded example be useful for a future project use case, and what additional privacy or interoperability requirements should it exercise?

Reproduction and artifacts

With Node 22 or newer, run npm ci --ignore-scripts followed by npm test from example/. Inspect evidence/report.json and its native artifact links. The report identifies the exact dependencies and runtime. The review page displays the captured run. Copy the packet before rerunning: native timestamps and run IDs change, and evidence is regenerated. Source, lockfile and checksums are included. No outreach has been sent.

References

[1] NCCoE concept paper on software and AI agent identity and authorization

[2] NCCoE summary of stakeholder comments

[3] NCCoE current project and rolling feedback route

FootPrint.js source

HCIFootprint source